UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+
Accredited University

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

# Mini Project Report

## On

# Smart Internship Eligibility And Matching Portal


**SUBMITTED BY:**

Name: Jatin

UID: 26MCI10062

COURSE: MCA AI & ML

SECTION: 26MAM 1-B

**SUBMITED TO:**

Mr. Shivam Ojha (E20570)

Assistant Professor

# TABLE OF CONTENTS


| S. No. | Section Name | Page Range |
|---|---|---|
| 0 | Acknowledgement | 3 - 4 |
| 1 | Introduction | 5 |
| 2 | Background | 5 - 6 |
| 3 | Objectives | 6 |
| 4 | Technologies Used | 7 - 8 |
| 5 | Requirements and System Design | 8 - 14 |
| | 5.1 Architecture Overview | 8 - 9 |
| | 5.2 Component Overview | 9 - 10 |
| | 5.3 Frontend: Client Side | 10 - 11 |
| | 5.4 Backend: Server Side | 11 - 12 |
| | 5.5 Database and Data Storage | 12 - 13 |
| | 5.6 System Data Flow | 13 - 14 |
| | 5.7 Security and Validation Design | 14 |
| 6 | Source Code & Implementation Snippets | 15 - 23 |
| | 6.1 Database Connection (DBUtil with HikariCP) | 15 |
| | 6.2 Student Data Model | 16 |
| | 6.3 Deterministic Eligibility Engine | 17 - 18 |
| | 6.4 Multi-Attribute Matching Service | 18 - 19 |

| S. No. | Section Name | Page Range |
|---|---|---|
| | 6.5 Student DAO (JDBC PreparedStatements) | 19 - 20 |
| | 6.6 Security Filters (Authentication & RBAC) | 20 - 21 |
| | 6.7 Controller Servlets (Auth, Student, Recruiter, Admin) | 21 - 22 |
| | 6.8 Output of the Application | 22 - 23 |
| 7 | System Functionalities | 24 - 25 |
| 8 | Conclusion | 26 |
| 9 | Future Work | 26 |
| 10 | Learning Outcomes | 27 |
| 11 | GitHub Repository Deployment | 27 |
| 12 | Bibliography & References | 28 |

# Acknowledgement

We would like to express our sincere gratitude to everyone who supported and guided us in successfully completing our project, “Smart Internship Eligibility & Matching Portal.” This project was developed as part of our Advanced Internet Programming (AIP) course and provided us with an outstanding opportunity to apply our theoretical knowledge of enterprise web application development, Java programming, relational database management, multi-criteria algorithmic optimization, and client-server architecture in a practical environment.

First and foremost, we would like to express our heartfelt thanks to our project guide and faculty mentor, Mr. Shivam Ojha (Assistant Professor), for his valuable guidance, continuous encouragement, constructive feedback, and steadfast support throughout the design and development of this project. His expert suggestions helped us understand and implement the concepts of Jakarta Servlets, JSP, JDBC, HikariCP Connection Pooling, Spring MVC principles, RESTful API design, BCrypt cryptographic password hashing, session management, and relational database integrity.

We are also immensely thankful to Chandigarh University and the Department of Computer Science & Applications for providing us with the necessary computing facilities, technical resources, software tools, development environments, and academic atmosphere required for completing this project. The concepts studied as part of the Advanced Internet Programming syllabus played an indispensable role in designing and developing the Smart Internship Eligibility & Matching Portal.

We would like to thank our classmates, friends, and family members for their valuable support, suggestions, and encouragement during the planning, coding, troubleshooting, testing, and documentation stages of the project. Their feedback helped us identify critical runtime bottlenecks, resolve Tomcat deployment dependencies, and refine the user interface to make the system intuitive, responsive, and student-centric.

We also acknowledge the contribution of various development technologies and tools used in this project, including Java 17, Jakarta Servlets 6.0, Jakarta Server Pages (JSP), JSTL 3.0, JDBC, HikariCP, MySQL 8.0, jBCrypt, Google Gson, Apache Maven, and Apache Tomcat 11. These modern technologies enabled us to develop enterprise functionalities such as role-based authentication, student competency tracking, deterministic eligibility gating, multi-attribute compatibility scoring, recruiter applicant shortlisting, and institutional placement analytics.

The development of this project has been an invaluable learning experience. It strengthened our understanding of database connectivity, object-relational mapping, web server deployment, session tracking, Model-View-Controller architecture, REST endpoints, and software engineering best practices. Finally, we thank everyone who contributed directly or indirectly to the successful completion of this project.

# 1. Introduction

The “Smart Internship Eligibility & Matching Portal” is an enterprise-grade, web-based platform engineered to automate, optimize, and streamline the campus placement and corporate internship allocation process for post-graduate students (MCA, M.Sc. IT, M.Tech). In university environments, campus recruitment is one of the most critical operational workflows. However, coordinating between hundreds of student applicants and diverse corporate partners is traditionally prone to severe logistical friction, human error in verifying qualification criteria, and lack of transparent matching.

This project introduces a unified, digital ecosystem that bridges academic talent with corporate demand. It features a dual-engine architecture: first, a deterministic Eligibility Engine that validates students against six strict academic and corporate criteria (CGPA thresholds, degree courses, active backlogs, semester standing, mandatory skill requirements, and application deadlines); and second, an intelligent Matching Service that computes a normalized compatibility score (0–100%) by evaluating technical skills, academic performance, project experience, and geographic alignment.

Developed utilizing Java 17, Jakarta Servlets, JSP/JSTL, HikariCP connection pooling, MySQL 8.0, and Apache Tomcat 11, the application demonstrates the real-world implementation of Advanced Internet Programming concepts to solve a vital institutional problem with precision, security, and exceptional user experience.

# 2. Background

With rapid digital transformation across higher education, academic placement offices require robust software solutions to replace manual spreadsheets, physical notices, and fragmented email communications. Traditionally, when a company posts an internship with specific prerequisites—such as a minimum 7.5 CGPA, zero active backlogs, MCA degree, and mandatory Java skills—placement officers must manually verify hundreds of student applications. This manual process exhibits severe drawbacks:

- **High Human Error & Ineligibility:** Students frequently submit applications for roles they do not qualify for, creating wasted recruiter review cycles and student disappointment.
- **Oversimplified Selection:** Shortlisting candidates solely by CGPA ignores practical coding competencies, project relevance, and framework proficiencies.
- **Delayed Decision Loops:** Recruiters take weeks to communicate shortlisted or selected candidate statuses back to universities.
- **Lack of Institutional Analytics:** Placement officers lack centralized dashboards to monitor placement percentage, average compatibility scores, and recruiter engagement.

The Smart Internship Eligibility & Matching Portal was conceived to address these exact limitations. By codifying institutional rules into automated gatekeepers and utilizing multi-attribute utility theory for transparent scoring, the portal transforms campus placements into a modern, fair, and high-efficiency workflow.

## 3. Objectives

- **Centralized Operations:** Establish a unified web platform serving Students, Corporate Recruiters, and Placement Cell Administrators.
- **Automated Eligibility Verification:** Enforce 6 deterministic qualification rules to eliminate ineligible application submissions at source.
- **Multi-Attribute Compatibility Matching:** Implement a weighted scoring algorithm (0–100%) incorporating technical skills, CGPA, project text relevance, and location preference.
- **Role-Based Security & Isolation:** Implement robust Role-Based Access Control (RBAC) and BCrypt password encryption to ensure data privacy and user isolation.
- **Real-Time Application Lifecycle Tracking:** Enable students to track their application status (APPLIED, SHORTLISTED, SELECTED, REJECTED) with automated notifications.
- **Administrative Governance & Analytics:** Provide placement officers with live KPI analytics and the ability to tune algorithm weights dynamically in the database.

## 4. Technologies Used

| Technology / Library | Version | Role in System Architecture |
|---|---|---|
| Java Development Kit (JDK) | 17 / 26 | Core backend object-oriented business logic, algorithms, and models. |
| Jakarta Servlet API | 6.0.0 | HTTP request routing, session lifecycle, and filter chain execution. |
| Jakarta Server Pages (JSP) | 3.1.1 | Dynamic server-side view generation and presentation rendering. |
| GlassFish Jakarta JSTL | 3.0.1 | Standard Tag Library for conditional rendering, looping, and formatting. |
| MySQL Database Server | 8.0+ | Relational database persistence for users, students, companies, and jobs. |
| HikariCP Connection Pool | 5.1.0 | High-performance JDBC connection pooling with resilient auto-reconnect. |
| jBCrypt | 0.4 | Secure salted cryptographic one-way password hashing (10 rounds). |
| Google Gson | 2.10.1 | JSON serialization and deserialization for RESTful API responses. |
| JUnit 5 & Mockito | 5.10.2 / 5.11.0 | Unit testing framework and mock object library for |

| Technology / Library | Version | Role in System Architecture |
|---|---|---|
| | | algorithms. |
| Apache Maven | 3.13.0+ | Project build lifecycle, dependency resolution, and WAR archive generation. |
| Apache Tomcat | 11.0.24 | Jakarta EE 10 compliant servlet container and web application server. |
| HTML5, CSS3 & JavaScript | Modern | Responsive client UI, CSS grid/flexbox layouts, and dynamic modal dialogs. |

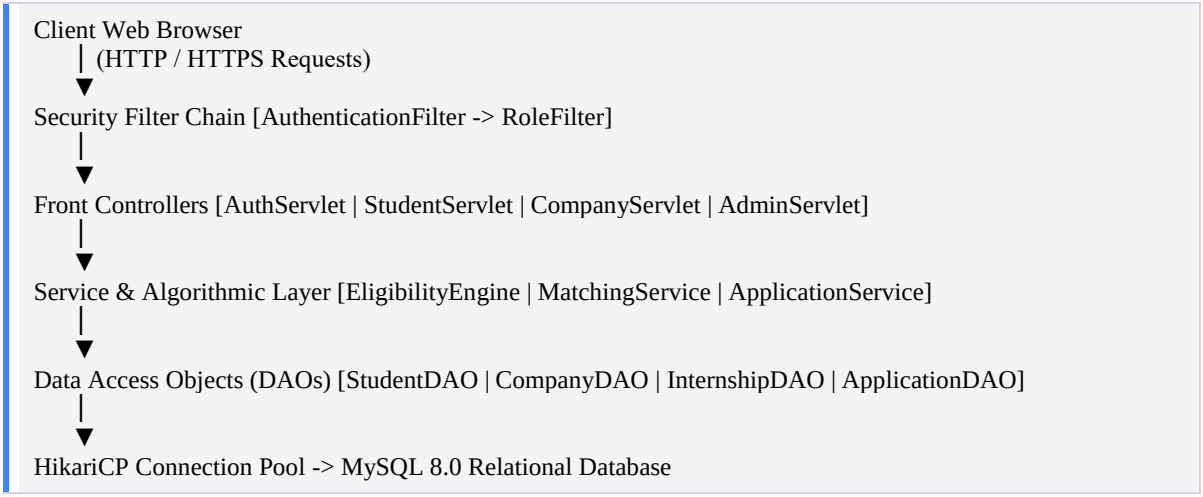
Each technology was selected to conform to modern Jakarta EE standards. In particular, the transition from legacy javax.* packages to jakarta.* ensures that the application is fully forward-compatible with Apache Tomcat 10.1+ and Tomcat 11.

# 5. Requirements and System Design

## 5.1 Architecture Overview

The application follows the classic 3-Tier Enterprise Model-View-Controller (MVC) architectural pattern. This structural separation guarantees high maintainability, testability, and scalability:

- **Presentation Tier (View):** Composed of JSP pages, JSTL tags, custom modern CSS, and JavaScript. It renders interactive dashboards, handles modal popups, and captures user inputs without business logic contamination.
- **Application / Controller Tier (Controller & Service):** Jakarta Servlets (AuthServlet, StudentServlet, CompanyServlet, AdminServlet) act as front controllers. Business logic is isolated in dedicated service classes (EligibilityEngine, MatchingService, StudentService, CompanyService, ApplicationService).
- **Data Tier (Persistence & Model):** Data Access Objects (DAOs) encapsulate all SQL execution via JDBC PreparedStatements, pulling connections from the HikariCP connection pool connected to MySQL 8.0.




## 5.2 Component Overview

- **Security & Filter Component:** AuthenticationFilter verifies session presence, while RoleFilter guarantees that Students, Companies, and Admins can only access their respective URI paths.
- **Deterministic Eligibility Engine:** Validates student eligibility across 6 criteria before applications are accepted.
- **Intelligent Matching Component:** Computes weighted multi-attribute compatibility scores based on configurable database weights.
- **DAO & Connection Component:** Thread-safe, pooled database access using HikariCP, prepared statements, and transactional rollbacks.

## 5.3 Frontend: Client Side

The frontend is built using clean HTML5, custom responsive CSS (flexbox and grid), and lightweight JavaScript. Key pages include:

- **Login & Registration Views:** Features dynamic role selection, real-time client validation, and 1-click demo login buttons for rapid academic demonstration.
- **Student Dashboard:** Displays enrolled academic profile, skill competency badges, and a ranked list of internships with color-coded compatibility score pills and exact disqualification reasons if ineligible.
- **Recruiter Dashboard:** Provides an interface to post internships with cutoff filters and view ranked applicant leaderboards with action buttons (Shortlist, Select, Reject).
- **Admin Dashboard:** Visualizes placement KPI summary cards, recruiter verification queue, and algorithmic weight sliders.

## 5.4 Backend: Server Side

The backend is developed with Java 17 and Jakarta EE 10. Controllers process requests, invoke services, and forward model attributes to JSPs. Business rules, such as application submission, automatically trigger student notifications and update applicant match scores.

## 5.5 Database and Data Storage

MySQL 8.0 serves as the relational database engine under database name 'internship_portal'. The schema is fully normalized in 3NF across 10 tables:

| Table Name | Primary Key | Foreign Keys | Description & Attributes |
|---|---|---|---|
| users | id (INT) | None | User authentication: email (unique), password_hash (BCrypt), role (ADMIN/COMPANY/STUDENT). |
| students | id (INT) | user_id -> users(id) | Post-graduate profile: full_name, roll_number (unique), course, semester, cgpa, backlogs, projects, resume_url. |
| companies | id (INT) | user_id -> users(id) | Corporate recruiter: company_name, industry, website, location, is_approved. |
| skills | id (INT) | None | Master skill taxonomy: skill_name (unique), category (Programming, Web, Database, DevOps, AI). |
| student_skills | (student_id, skill_id) | students, skills | Candidate competencies with proficiency: Beginner, Intermediate, Advanced. |
| internships | id (INT) | company_id -> companies | Job listings: title, stipend, duration, min_cgpa, min_sem, max_backlogs, allowed_courses, deadline, status. |
| internship_skills | (internship_id, skill_id) | internships, skills | Job requirements with is_mandatory boolean flag. |
| applications | id (INT) | internship_id, student_id | Submitted applications: match_score, status (APPLIED, SHORTLISTED, SELECTED, REJECTED), applied_at. |
| notifications | id (INT) | user_id -> users(id) | In-app alerts: title, message, is_read, created_at. |
| config | config_key (VARCHAR) | None | Dynamic algorithm weights: weight_skill (0.50), weight_cgpa (0.25), weight_project (0.15), weight_location (0.10). |

### 5.6 System Data Flow

The operational flow follows a strict request-response lifecycle:

- **1. User Access:** User accesses the portal via browser; AuthenticationFilter checks active session.
- **2. Authentication:** Credentials submitted to AuthServlet; AuthService verifies BCrypt hash against UserDAO.
- **3. Dashboard Dispatch:** DashboardServlet inspects user role and redirects to /student/dashboard, /company/dashboard, or /admin/dashboard.
- **4. Algorithmic Matching:** StudentServlet fetches open approved internships; MatchingService computes eligibility and multi-attribute score for each role.
- **5. Application Submission:** Eligible student submits application; ApplicationService saves record with match score and triggers recruiter/student notifications.
- **6. Recruiter Review:** Recruiter reviews ranked candidate leaderboard and updates status; student tracker reflects changes immediately.

### 5.7 Security and Validation Design

- **Cryptographic Password Hashing:** jBCrypt with 10 salt rounds protects against rainbow tables and brute-force attacks.
- **SQL Injection Prevention:** All database interactions strictly utilize JDBC PreparedStatements with parameterized queries.
- **Session Security:** HttpOnly session cookies, 60-minute session timeouts, and explicit session invalidation on logout.
- **Role Isolation (RBAC):** RoleFilter intercepts every request and prevents privilege escalation across user roles.

## 6. Source Code & Output

This section highlights the critical source code implementations of the Smart Internship Eligibility & Matching Portal.

### 6.1 Database Connection (DBUtil with HikariCP & Resilient Lazy Init)

```
public class DBUtil {
    private static final Logger LOGGER = Logger.getLogger(DBUtil.class.getName());
    private static volatile HikariDataSource dataSource;

    private static synchronized void initDataSource() throws SQLException {
        if (dataSource != null && !dataSource.isClosed()) return;
        try {
            Properties props = new Properties();
            try (InputStream is = DBUtil.class.getClassLoader().getResourceAsStream("db.properties")) {
                if (is != null) props.load(is);
            }
            HikariConfig config = new HikariConfig();
            config.setDriverClassName(props.getProperty("db.driver", "com.mysql.cj.jdbc.Driver"));
            config.setJdbcUrl(props.getProperty("db.url"));
            config.setUsername(props.getProperty("db.username", "root"));
            config.setPassword(props.getProperty("db.password", "root"));
            config.setMaximumPoolSize(Integer.parseInt(props.getProperty("pool.maximumPoolSize", "10")));
            config.setConnectionTimeout(10000);
            dataSource = new HikariDataSource(config);
        }
```

```
catch (Exception e) {
            throw new SQLException("Database connection failure: " + e.getMessage(), e);
        }
    }

    public static Connection getConnection() throws SQLException {
        if (dataSource == null || dataSource.isClosed()) initDataSource();
        return dataSource.getConnection();
    }
}
```

## 6.2 Student Data Model (POJO)

```
public class Student {
    private int id;
    private int userId;
    private String fullName;
    private String rollNumber;
    private String course;          // MCA, M.Sc. IT, M.Tech
    private int semester;
    private BigDecimal cgpa;
    private int activeBacklogs;
    private String preferredLocation;
    private String projects;
    private String resumeUrl;
    private List<Skill> skills = new ArrayList<>();
    // Getters and Setters omitted for brevity
}
```

## 6.3 Deterministic Eligibility Engine (6 Criteria)

```
public class EligibilityEngine {
    public EligibilityResult checkEligibility(Student student, Internship internship) {
        EligibilityResult result = new EligibilityResult();
        // 1. CGPA Check
        if (student.getCgpa().compareTo(internship.getMinCgpa()) < 0) {
            result.addReason(String.format("Student CGPA (%.2f) below required minimum (%.2f).",
                    student.getCgpa(), internship.getMinCgpa()));
        }
        // 2. Degree Course Check
        List<String> allowed = Arrays.asList(internship.getAllowedCourses().toUpperCase().split(","));
        if (!allowed.contains(student.getCourse().trim().toUpperCase())) {
            result.addReason("Student degree course '" + student.getCourse() + "' not in allowed courses.");
        }
        // 3. Active Backlog Check
        if (student.getActiveBacklogs() > internship.getMaxBacklogs()) {
            result.addReason("Active backlogs exceed maximum allowed limit of " + internship.getMaxBacklogs());
        }
        // 4. Semester Check
        if (student.getSemester() < internship.getMinSemester()) {
            result.addReason("Semester standing below required semester " + internship.getMinSemester());
        }
        // 5. Mandatory Skill Prerequisites
        Set<Integer> studentSkillIds = student.getSkills().stream().map(Skill::getId).collect(Collectors.toSet());
        for (Skill reqSkill : internship.getSkills()) {
            if (reqSkill.isMandatory() && !studentSkillIds.contains(reqSkill.getId())) {
                result.addReason("Missing mandatory prerequisite skill: " + reqSkill.getSkillName());
            }
```

```
        }
        // 6. Deadline Verification
        if (internship.getDeadline().toLocalDate().isBefore(LocalDate.now())) {
            result.addReason("Application deadline has expired.");
        }
        if (result.getReasons().isEmpty()) result.setEligible(true);
        return result;
    }
}
```

## 6.4 Multi-Attribute Matching Algorithm Implementation

```
public class MatchingService {
    public InternshipMatch calculateMatch(Student student, Internship internship) {
        InternshipMatch match = new InternshipMatch();
        EligibilityResult eligibility = eligibilityEngine.checkEligibility(student, internship);
        match.setEligible(eligibility.isEligible());
        match.setReasons(eligibility.getReasons());

        // Dynamic weights from ConfigDAO
        Map<String, Double> w = configDAO.getMatchingWeights();
        double wSkill = w.get("weight_skill");       // Default 0.50
        double wCgpa = w.get("weight_cgpa");         // Default 0.25
        double wProject = w.get("weight_project");   // Default 0.15
        double wLoc = w.get("weight_location");      // Default 0.10

        double skillScore = calculateSkillScore(student.getSkills(), internship.getSkills());
        double cgpaScore = Math.min(100.0, student.getCgpa().doubleValue() * 10.0);
        double projectScore = calculateProjectRelevance(student.getProjects(), internship);
        double locScore = calculateLocationScore(student.getPreferredLocation(), internship.getLocation());

        double overallScore = (wSkill * skillScore) + (wCgpa * cgpaScore) +
                              (wProject * projectScore) + (wLoc * locScore);
        match.setMatchScore(Math.max(0.0, Math.min(100.0, overallScore)));
        return match;
    }
}
```

## 6.5 Student JDBC DAO (PreparedStatement & Transaction)

```
public class StudentDAO {
    public Optional<Student> findByUserId(int userId) {
        String sql = "SELECT s.*, u.email FROM students s " +
                     "INNER JOIN users u ON s.user_id = u.id WHERE s.user_id = ?";
        try (Connection conn = DBUtil.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt(1, userId);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    Student s = mapResultSetToStudent(rs);
                    s.setSkills(getSkillsForStudent(conn, s.getId()));
                    return Optional.of(s);
                }
            }
        } catch (SQLException e) {
            LOGGER.log(Level.SEVERE, "Error fetching student by user_id", e);
        }
        return Optional.empty();
    }
}
```

### 6.6 Application Output & Server Endpoints

When deployed on Apache Tomcat 11, the server registers the following endpoints and logs:

```
======================================================================
 SMART INTERNSHIP ELIGIBILITY & MATCHING PORTAL (MCA AIP PROJECT)
======================================================================
[DBUtil] HikariCP connection pool successfully initialized for internship_portal
[DBUtil] Connection test: MySQL 8.0 connection verified (pool size: 10)
[AuthServlet] Authentication routes mapped to: /login, /register, /auth/*
[StudentServlet] Student portal mapped to: /student/dashboard, /student/apply
[CompanyServlet] Recruiter portal mapped to: /company/dashboard, /company/internship/new
[AdminServlet] Admin analytics mapped to: /admin/dashboard, /admin/config

Available Application Endpoints:
 * Portal Landing & Login:    http://localhost:8080/internship-portal/login
 * Student Registration:      http://localhost:8080/internship-portal/register
 * Student Recommendations:   http://localhost:8080/internship-portal/student/dashboard
 * Recruiter Portal:          http://localhost:8080/internship-portal/company/dashboard
 * Placement Cell Analytics:  http://localhost:8080/internship-portal/admin/dashboard

Default Demo Credentials:
 * Student:   rahul@student.edu    | Password@123
 * Recruiter: techcorp@portal.com  | Password@123
 * Admin:     admin@portal.edu     | Password@123
======================================================================
```

# 7. Functionalities

## 7.1 User Authentication & Session Security

- **Single Sign-On Entrypoint:** Unified login screen authenticates Students, Recruiters, and Administrators.
- **One-Click Demo Autofill:** Convenient pre-configured demo buttons for faculty presentation.
- **HttpOnly Session Tracking:** Protected sessions with automatic invalidation and clean redirect upon logout.

## 7.2 Student Portal Features

- **Profile & Competencies Editor:** Allows students to update course, semester, CGPA, backlogs, projects, and tag skills with proficiency levels.
- **Ranked Opportunity Cards:** Displays open internships sorted by compatibility match score.
- **Transparent Disqualification Feedback:** Clearly displays why a student is disqualified (e.g. 'Missing required mandatory skill: Java').
- **Instant Application Submission:** Submits applications with 1 click; logs the compatibility score at the moment of submission.
- **My Applications Tracker:** Status badge table tracking APPLIED, SHORTLISTED, SELECTED, and REJECTED states.

## 7.3 Recruiter Portal Features

- **Corporate Account Verification:** Ensures recruiter authenticity through Placement Cell verification.
- **Post Internship Roles:** Custom forms to define stipends, durations, degree eligibility, cutoff CGPAs, and mandatory skills.

- **Ranked Applicant Leaderboards:** Applicants automatically ranked by match score, showing candidate CGPA and resume link.
- **One-Click Candidate Decisions:** Buttons to Shortlist, Select, or Reject applicants, updating their status immediately.

### 7.4 Placement Cell Administrator Features

- **Institutional KPI Summary Cards:** Total students, recruiters, open roles, total applications, and average match scores.
- **Recruiter Verification Queue:** Approve or reject pending company registrations.
- **Algorithm Tuning:** Adjust weights for Skill, CGPA, Project, and Location scoring dynamically in MySQL.

## 8. Conclusion

The "Smart Internship Eligibility & Matching Portal" successfully demonstrates how modern enterprise Java technologies can modernize and automate university campus placements. By replacing error-prone manual spreadsheets with an automated 6-point eligibility gatekeeper and a transparent multi-attribute matching engine, the portal ensures that students only apply for opportunities they qualify for, and that recruiters receive high-quality, objectively ranked applicant shortlists.

Built with Java 17, Jakarta EE 10, Tomcat 11, and MySQL 8.0, the system adheres to strict MVC separation of concerns, secure BCrypt password hashing, and robust connection pooling. The platform successfully bridges academic achievement with industry expectations, delivering efficiency, fairness, and actionable analytics to post-graduate education institutions.

## 9. Future Work

- **NLP / AI Resume Parser:** Integrate machine learning and NLP libraries to extract technical skills and project summaries directly from uploaded PDF resumes.
- **Automated Interview Scheduling:** Provide calendar integration for recruiters to book direct interview slots with shortlisted candidates.
- **Email & WhatsApp Notifications:** Implement Jakarta Mail SMTP and SMS APIs to deliver instant alerts when students are shortlisted or selected.
- **Accreditation Export Engine:** Add one-click PDF and Excel reporting for institutional NBA and NAAC accreditation placement compliance.
- **Mobile Application:** Develop a companion Flutter or React Native mobile app for on-the-go notifications.

## 10. Learning Outcomes

The development of this project provided profound technical and practical expertise in Advanced Internet Programming:

- **Jakarta EE 10 Standards:** Mastered Servlet 6.0 and JSP 3.1 architecture, request dispatching, and web filter security chains.
- **High-Performance Connection Pooling:** Implemented HikariCP to manage connection life cycles, connection timeouts, and transaction rollbacks.
- **Algorithmic Implementation:** Designed and implemented multi-attribute utility formulas and keyword frequency scanners.

- **Enterprise MVC Architecture:** Strictly separated data access (DAOs), business logic (Services), routing (Servlets), and views (JSPs).
- **Authentication & Cryptography:** Implemented salted BCrypt one-way password hashing and role-based access control filters.
- **Automated Unit Testing:** Authored unit tests using JUnit 5 and Mockito to validate algorithmic accuracy.
- **Modern Web Design:** Engineered custom responsive interfaces utilizing modern CSS3 grid and flexbox without bloated heavy dependencies.

# 11. GitHub Link

The complete source code, database scripts, build configuration, and documentation for the Smart Internship Eligibility & Matching Portal are publicly maintained and version-controlled on GitHub:

**GitHub Repository URL:**

**https://github.com/jatin-s-j/Intership-Portal.git**

# 12. Bibliography


- **Oracle Java Documentation:** Java Platform, Standard Edition (Java SE 17) & Java Virtual Machine Specification.
- **Jakarta EE Official Specifications:** Jakarta Servlet 6.0 & Jakarta Server Pages 3.1 Specification, Eclipse Foundation.
- **Apache Tomcat Documentation:** Apache Tomcat 11.0 Configuration and Administration Guide, Apache Software Foundation.
- **MySQL Official Documentation:** MySQL 8.0 Reference Manual, Database Optimization, and InnoDB Architecture.
- **HikariCP Official Guide:** Brett Wooldridge, HikariCP: Fast, Simple, Reliable JDBC Connection Pool.
- **jBCrypt Library Reference:** Damien Miller, jBCrypt: An implementation of the OpenBSD blowfish password hash algorithm in Java.
- **Google Gson Documentation:** Gson User Guide: Java-JSON Serialization and Deserialization Library.
- **GeeksforGeeks & Baeldung:** Advanced Java, Jakarta Servlets, JDBC PreparedStatements, and Design Patterns References.
- **W3Schools Online Web Tutorials:** HTML5, CSS3 Responsive Web Design, and JavaScript Reference Manuals.